

Introduction to Artificial Intelligence IP 1

Solving Sokoban

Zac Serocki

February 6, 2026

1 FORMULATION OF SOKOBAN

To represent the game state, a tuple of tuples of characters was used, ensuring that states are hashable and compatible with memoization in search algorithms. When generating successor states, the board is temporarily converted into a list of lists for efficient modification and then converted back to an immutable structure. The implementation preserves the original notation for walls (%) and boxes (b for boxes off a button and B for boxes on a button). The player is represented as p when standing on a free space and P when standing on a button. While this distinction provides minimal functional benefit, it maintains representational continuity and enables concise checks using the `.isupper()` method to identify goal-related tiles.

Actions are represented as tuples of index changes, allowing a uniform and reusable movement representation across action validation, state transitions, and heuristic evaluation. Illegal actions are filtered early using a shared validation routine, reducing unnecessary state expansions. State transitions are handled using a recursive movement function that cleanly supports box pushing while preserving tile semantics through character case. The heuristic combines a breadth-first search-based distance from boxes to the nearest goal with deadlock detection for irrecoverable box positions, assigning infinite cost to unsolvable states to aggressively prune the search space. Overall, the code follows Clean Code principles, with each function performing a single well-defined task, resulting in a modular, readable, and reusable implementation.

2 A* HEURISTICS

For Heuristics I choose to find the To determine the distance from every box to the goal, I used breadth first search to find the nearest goal and consider the walls of the map. This heuristic is admissible because it does not account for multiple boxes potentially targeting the same goal or the additional moves required to push a box to its goal. As a result, the estimated cost is always less than or equal to the true cost.

A secondary heuristic detects soft locks, ensuring that the algorithm does not waste time expanding nodes that are unwinnable. This heuristic checks for boxes that are irreversibly trapped, such as those stuck in corners or against walls in positions from which they cannot reach a goal. By assigning these states an infinite cost, the search efficiently prunes unsolvable branches. This is admissible because soft-locked states are truly unsolvable—they cannot reach the goal regardless of the number of moves. Assigning them infinite cost accurately reflects that the true cost is infinite (or undefined).

Applying the max function on the individual heuristics is strictly dominant than each individual heuristic; this is admissible since if both heuristics are admissible individually, then the maximum of the heuristics is admissible.

Algorithm 1 Soft Lock

```
function SOFT_LOCK(state)
  for i,line in state do
    for j, char in line do
      if char  $\neq$  b then continue

      pos = (i,j)
      actions = available_actions((i,j))           ▷ Checks if the block is in a corner
      sum = (0,0)
      for action in actions do
        sum = sum + action
      if  $\forall \text{abs}(sum_i) = 1$  then return  $\infty$ 

      if right(pos) = "b" then           ▷ Checks if there are 2 blocks next to each other
        if down(pos) = "%" and down(right(pos)) = "%" then return  $\infty$ 
        if up(pos) = "%" and up(right(pos)) = "%" then return  $\infty$ 
      if down(pos) = "b" then
        if left(pos) = "%" and left(down(pos)) = "%" then return  $\infty$ 
        if right(pos) = "%" and right(down(pos)) = "%" then return  $\infty$ 
```

3 RESULTS AND ANALYSIS

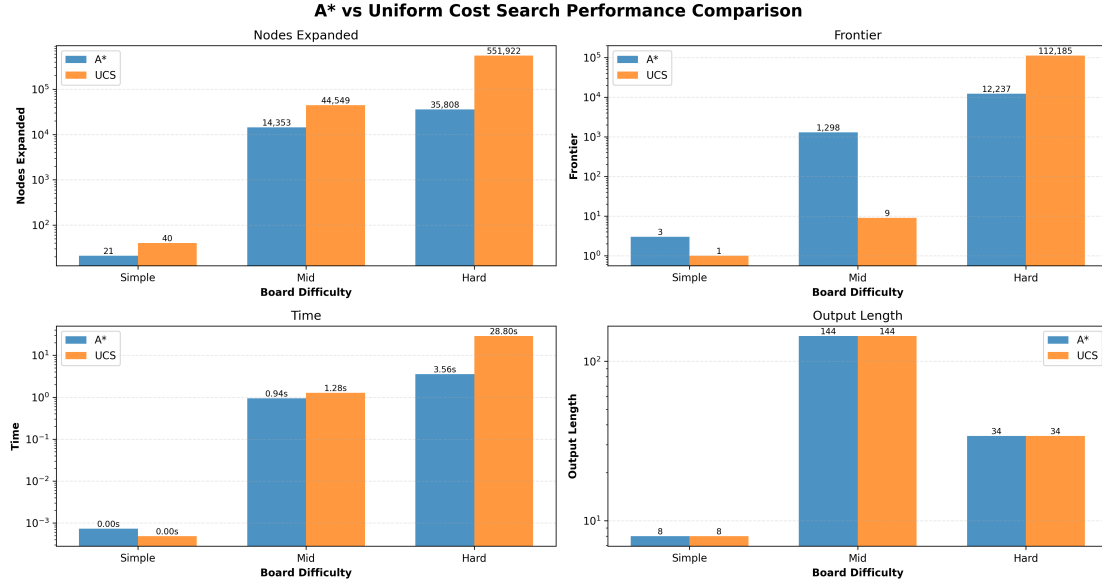


Figure 3.1: Performance comparison between A* and Uniform Cost Search (UCS) across increasing board difficulty. On the hard puzzle, A* expands approximately 15.5 times fewer nodes than UCS and maintains a frontier with over 100,000 fewer nodes, demonstrating substantially better scalability and search efficiency while producing solutions of identical length. We see that the output length is the same which is consistent with and admissible heuristic.

Overall, A* outperformed UCS in most cases. For the simple board, the overhead of computing the heuristic caused A* to run slightly slower than UCS, despite expanding fewer nodes. However, as problem difficulty increased, the benefit of heuristic guidance became dominant, leading to significantly improved performance.

On the medium-difficulty board, A* expanded substantially fewer nodes than UCS, resulting in lower runtime. Although A* maintained approximately 1,300 more nodes in the frontier than UCS, this is likely due to the board's state-space capacity being reached earlier by UCS, which expanded over 44,500 nodes overall.

The performance gap is most pronounced on the hard puzzle. A* expanded approximately 15.5 times fewer nodes than UCS and took 8.6 times less time. Additionally, A* maintained only about 10% of the frontier size of UCS, indicating much better memory efficiency. This advantage would be expected to grow further as puzzle difficulty increases.

Finally, both UCS and A* produced solutions of identical length across all inputs. This is an important result: while admissibility cannot be proven through empirical results alone, identical solution lengths indicate that the heuristic did not lead to suboptimal paths in practice.